EXPERIMENT: GENERATE TEXT EMBEDDINGS AND PERFORM SEMANTIC SIMILARITY SEARCH

Aim

To convert text documents and a user query into numerical vector representations (TF-IDF text embeddings), calculate cosine similarity, and retrieve documents in order of relevance.

Software / Requirements

Python 3.x, scikit-learn library, and any Python IDE or terminal (such as VS Code). Install the required library using `pip install scikit-learn`.

Procedure

1. Open VS Code or a Python editor and create a file named `semantic_similarity_search.py`.

2. Import `TfidfVectorizer` and `cosine_similarity` from scikit-learn.

3. Prepare a small collection of text documents that will act as the searchable document database.

4. Create a TF-IDF vectorizer and convert the documents into numerical vectors. These vectors are the text embeddings used in this experiment.

5. Convert the user's search query into a vector using the same fitted vectorizer.

6. Calculate cosine similarity between the query vector and each document vector.

7. Sort the similarity scores in descending order and display the ranked documents.

8. Run the program and observe the terminal output. The highest score indicates the closest match under this TF-IDF representation.

Program Code

```
# Generate text embeddings and perform semantic similarity search
# This experiment uses TF-IDF vectors as lightweight text embeddings
# and cosine similarity to rank the most relevant documents.

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

documents = [
    "Machine learning enables computers to learn patterns from data.",
    "Deep learning uses neural networks with multiple layers.",
    "A car engine converts fuel energy into mechanical power.",
    "Natural language processing helps computers understand human language.",
    "Python is a popular programming language for data science."
]

query = "How can computers learn from information and data?"

# Convert documents and query into numerical vector representations.
vectorizer = TfidfVectorizer(stop_words="english")
doc_vectors = vectorizer.fit_transform(documents)
query_vector = vectorizer.transform([query])

# Calculate cosine similarity between the query and every document.
scores = cosine_similarity(query_vector, doc_vectors).flatten()

# Display results from highest to lowest similarity.
ranked_indices = scores.argsort()[::-1]

print("QUERY:", query)
print("\nSEMANTIC SIMILARITY SEARCH RESULTS")
for rank, index in enumerate(ranked_indices, start=1):
    print(f"{rank}. Similarity: {scores[index]:.4f}")
    print(f"   Document: {documents[index]}")
```

Execution Screenshot

The following screenshot-style capture contains the output produced by running the program in this environment.

Python Terminal — Experiment Execution

```
QUERY: How can computers learn from information and data?

SEMANTIC SIMILARITY SEARCH RESULTS
1. Similarity: 0.6218
   Document: Machine learning enables computers to learn patterns from data.
2. Similarity: 0.1863
   Document: Python is a popular programming language for data science.
3. Similarity: 0.1493
   Document: Natural language processing helps computers understand human language.
4. Similarity: 0.0000
   Document: A car engine converts fuel energy into mechanical power.
5. Similarity: 0.0000
   Document: Deep learning uses neural networks with multiple layers.
Spreadsheet runtime warmup failed during python startup
Traceback (most recent call last):
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/patches/warm_spreadsheet_runtime_on_startup.py
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/spreadsheet_warmup.py", line 785, in warm_spre
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/spreadsheet_warmup.py", line 718, in _warm_fea
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/spreadsheet_warmup.py", line 509, in _warm_dra
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/generated/interface/models.py", line 32861, in
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/rpc/remote.py", line 764, in __call__
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/rpc/client.py", line 175, in call
artifact_tool.rpc.client.RemoteError: Element ordering requires an owning element collection.

Process finished with exit code 0.
```

Result

The program executed successfully. It generated TF-IDF text vectors for the document collection, calculated cosine similarity for the input query, and displayed the documents in descending similarity order. The document with the highest score is returned as the most relevant result for the query.

Note: TF-IDF vectors are useful numerical text representations, but they mainly capture word importance and overlap. They are not contextual embeddings like those produced by transformer models (for example, Sentence-BERT). Therefore, this experiment demonstrates a basic similarity-search pipeline; semantically equivalent sentences with very different wording may not always rank highly.